INVENTORY MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

SABARINATH R	**2403811720521047**
SAKTHI DARSHAN K	**2403811720521049**
SUDHAN A	**2403811720521053**
SURIYAA R	**2403811720521054**

in partial fulfillment of the requirements for the award degree of

Bachelor of Technology

CGB1221 - DATABASE MANAGEMENT SYSTEM

DEPARTMENT OF INFORMATION TECHNOLOGY

K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(AUTONOMOUS)

SAMAYAPURAM - 621112

MAY 2026

INVENTORY MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

SABARINATH R	**2403811720521047**
SAKTHI DARSHAN K	**2403811720521049**
SUDHAN A	**2403811720521053**
SURIYAA R	**2403811720521054**

in partial fulfillment of the requirements for the award degree of

Bachelor of Technology

CGB1221 - DATABASE MANAGEMENT SYSTEM

DEPARTMENT OF INFORMATION TECHNOLOGY

K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(AUTONOMOUS)

SAMAYAPURAM - 621112

MAY 2026

K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY
(AUTONOMOUS)
SAMAYAPURAM - 621112

BONAFIDE CERTIFICATE

The work embodied in the present project report entitled "**INVENTORY MANAGEMENT SYSTEM**" has been carried out by the students SABARINATH R, SAKTHI DARSHAN K, SUDHAN A, SURIYAA R, The work reported herein is original and we declare that the project is their own work, except where specifically acknowledged, and has not been copied from other sources or been previously submitted for assessment.

Date of Viva Voce: ……

Mr. A. MALARMANNAN
SUPERVISOR,
Assistant Professor,
Information Technology,
K.Ramakrishnan College Of Technology(Autonomous),
Samayapuram-621112

Dr. C. SHYAMALA
HEAD OF THE DEPARTMENT,
Associate Professor,
Information Technology,
K.Ramakrishnan College Of Technology(Autonomous),
Samayapuram-621112

INTERNAL EXAMINER **EXTERNAL EXAMNIER**

ABSTRACT

The Inventory Management System is a comprehensive database-driven solution designed to streamline the tracking of stock, suppliers, customers, and transactions for organizations of all sizes. This project addresses the common challenges faced by businesses, including data redundancy, inconsistent record-keeping, delayed information retrieval, and difficulty in generating meaningful reports. The system is built using MySQL as the backend database and PHP for API handling, with a modern, responsive frontend interface. The database schema consists of five core tables: Suppliers, Customers, Products, Transactions, and Inventory, all linked through primary and foreign key relationships to ensure data integrity and normalization. Constraints such as NOT NULL, UNIQUE, and CHECK validations enforce data accuracy at the database level. Key features include real-time inventory tracking with automatic stock level updates after every sale, purchase, return, or transfer transaction. The system provides low stock alerts, revenue analytics, transaction history, and comprehensive reporting capabilities. CRUD operations are supported for all entities, and a dashboard visualizes key metrics including total products, suppliers, customers, revenue, and low stock counts.

Keywords: Inventory Management, DBMS, MySQL, Stock Tracking, Transaction Management, CRUD Operations, Database Normalization.

ACKNOWLEDGEMENT

We thank our **Dr. N.Vasudevan**, Principal, for his valuable suggestions and support during the course of my research work.

We thank our **Dr. C. Shyamala,** Head of the Department, Information Technology, for her valuable suggestions and support during the course of my research work.

We wish to record my deep sense of gratitude and profound thanks to my Guide **Mr. A. Malarmannan,** Information Technology for his keen interest, inspiring guidance, constant encouragement with my work during all stages, to bring this thesis into fruition.

We are extremely indebted to our project coordinator **Mr. A. Malarmannan**, Information Technology, for his valuable suggestions and support during the course of my research work.

We also thank the faculty and non-teaching staff members of the Information Technology, K.Ramakrishnan College Of Technology, Trichy, for their valuable support throughout the course of my research work.

Finally, we thank our parents, friends and our well wishes for their kind support.

SIGNATURE

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Inventory Management is a critical business function that involves overseeing and controlling the ordering, storage, and usage of components that a company uses in the production of the items it sells, as well as the management of finished products that are available for sale. An effective inventory management system helps businesses track stock levels, manage suppliers, process customer transactions, and generate insightful reports for decision-making.

In today's competitive business environment, organizations require real-time visibility into their inventory status to prevent stockouts, reduce carrying costs, and improve customer satisfaction. Traditional manual methods using spreadsheets or paper records are prone to errors, data inconsistency, and delays in information retrieval.

This project presents a Database Management System (DBMS) based Inventory Management System that provides a structured approach to handle inventory data efficiently. The system leverages the power of relational databases to ensure data integrity, eliminate redundancy, and enable fast query processing.

1.2 PROBLEM DEFINITION

Many organizations struggle with the following challenges in their current inventory management practices:

1. **Data Redundancy and Inconsistency** – Storing the same information in multiple places leads to duplicate records and conflicting data.

2. **Poorly Structured Databases** – Lack of proper database design results in inefficient data retrieval and storage.

3. **Delayed Information Retrieval** – Manual searching through paper records or unorganized digital files causes significant delays.

4. **Difficulty in Generating Reports** – Creating meaningful reports for management decision-making becomes time-consuming and error-prone.

5. **No Real-time Stock Visibility** – Businesses cannot track current stock levels accurately, leading to stockouts or overstocking.

6. **Lack of Transaction Tracking** – Sales, purchases, and returns are not properly recorded and linked to inventory changes.

7. **Absence of Low Stock Alerts** – No automated notifications when products reach reorder levels.

1.3 OBJECTIVES OF THE PROJECT

The primary objectives of this Inventory Management System project are:

1. Design a Structured Database – Create a normalized relational database schema with appropriate tables, relationships, and constraints.

2. Maintain Accurate Records – Store and manage information about products, suppliers, customers, and transactions with high data integrity.

3. Reduce Data Redundancy – Implement normalization techniques to eliminate duplicate data storage.

4. Enable Fast Data Retrieval – Design efficient SQL queries and indexes for quick information access.

5. Provide Reliable Reports – Generate meaningful reports including revenue analysis, inventory status, and transaction history.

6. Implement Stock Tracking – Monitor inventory levels and automatically update stock after each transaction.

7. Ensure Data Security – Implement input validation and proper database constraints to maintain data quality.

1.4 SCOPE OF THE PROJECT

The scope of this Inventory Management System includes:

- **Supplier Management** – Adding, updating, deleting, and viewing supplier information including contact details and address.

- **Customer Management** – Maintaining customer profiles including name, email, phone, and purchase history.

- **Product Management** – Managing product details including name, category, price, quantity, and supplier linkage.

- **Transaction Management** – Recording sales, purchases, returns, and transfers with automatic stock updates.

- **Inventory Tracking** – Monitoring current stock levels with visual indicators for In Stock, Low Stock, and Out of Stock status.

- **Dashboard Analytics** – Displaying key metrics including total products, suppliers, customers, revenue, and low stock alerts.
- **Report Generation** – Providing transaction history and monthly revenue analysis.

1.5 DATABASE MANAGEMENT SYSTEM CONCEPTS

A **Database Management System (DBMS)** is software used to store, manage, and retrieve data efficiently. It acts as a bridge between users and the database, ensuring data is organized, secure, and easily accessible.

In the Inventory Management System, DBMS is used to manage data related to products, suppliers, customers, transactions, and inventory.

Key Concepts:

- **Database:** Organized collection of data stored in tables.
- **Table:** Structure consisting of rows (records) and columns (fields).
- **Primary Key:** Uniquely identifies each record in a table.
- **Foreign Key:** Establishes relationships between tables.
- **Constraints:** Rules like NOT NULL, UNIQUE, and CHECK to ensure data accuracy.
- **Normalization:** Organizing data to reduce redundancy and improve efficiency.
- **SQL (Structured Query Language):** Used to perform operations like INSERT, SELECT, UPDATE, and DELETE.

- **Data Integrity:** Ensures accuracy and consistency of data.
- **Transactions (ACID Properties):** Ensures reliable database operations.
- **Indexing:** Improves speed of data retrieval.

CHAPTER 2

LITERATURE SURVEY

2.1 EXISTING SYSTEM

Various inventory management approaches currently exist in the market:

Manual Systems (Paper-based):

- Records maintained in registers and logbooks
- Stock counted physically at intervals
- Transactions recorded manually
- Reports prepared using calculators and spreadsheets

Spreadsheet-based Systems (MS Excel/Google Sheets):

- Data stored in tabular format
- Basic formulas for calculations
- Limited data validation
- Prone to manual entry errors

Traditional Software Solutions:

- Desktop-based applications like Tally, SAP Business One
- Require installation on local machines
- Limited remote access capabilities
- Often expensive for small businesses

2.2 LIMITATIONS OF EXISTING SYSTEM

- **Data Redundancy** – Same data is stored multiple times across different sheets or files.
- **No Real-time Updates** – Stock levels do not update automatically after sales.
- **Poor Data Integrity** – No constraints are present to prevent invalid data entry.
- **Limited Querying** – Complex operations like joins and aggregations cannot be performed easily.
- **No Multi-user Support** – Only one user can access or edit the data at a time.
- **Manual Stock Calculation** – Stock calculations require manual formula updates.
- **No Historical Tracking** – Previous stock levels and transactions are not preserved.

2.3 PROPOSED SYSTEM

The proposed Inventory Management System addresses the limitations of existing systems through:

1. **Relational Database Design –** Normalized tables with proper relationships eliminate redundancy**.**
2. **Real-time Stock Updates –** Inventory levels automatically adjust after each transaction using database triggers and transactions.
3. **Data Integrity Constraints –** Primary keys, foreign keys, and CHECK constraints ensure valid data.
4. **Advanced Querying –** Complex SQL queries using JOINs, GROUP BY, and aggregate functions.

5. **Multi-user Access –** Concurrent access with proper transaction management.

6. **Automated Reporting** – Dynamic dashboard with revenue charts and transaction history.

7. **Status Indicators –** Visual badges for stock levels (In Stock, Low Stock, Out of Stock).

2.4 ADVANTAGES OF PROPOSED SYSTEM

- **Reduced Manual Effort** – Automated stock updates and calculations
- **Elimination of Data Duplication** – Normalized database design
- **Quick Report Generation** – Real-time analytics dashboard
- **Scalable Architecture** – Can handle growing data volumes
- **Data Accuracy** – Constraints and validations prevent errors
- **Easy Data Retrieval** – Search and filter functionality
- **Cost Effective** – Open source technologies (MySQL, PHP)
- **User Friendly** – Modern responsive web interface

CHAPTER 3

SYSTEM DESIGN

3.1 ENTITY RELATIONSHIP (ER) DIAGRAM

The ER diagram for the Inventory Management System consists of the following entities and relationships:

Entities:

1. **SUPPLIER** – Provides products to the organization
2. **CUSTOMER** – Purchases products from the organization
3. **PRODUCT** – Items managed in inventory
4. **TRANSACTION** – Records sales, purchases, returns, transfers
5. **INVENTORY** – Tracks current stock levels

Relationships:

- A SUPPLIER **supplies** many PRODUCTS (One-to-Many)
- A CUSTOMER **makes** many TRANSACTIONS (One-to-Many)
- A PRODUCT **is involved in** many TRANSACTIONS (One-to-Many)
- A PRODUCT **has one** INVENTORY record (One-to-One)

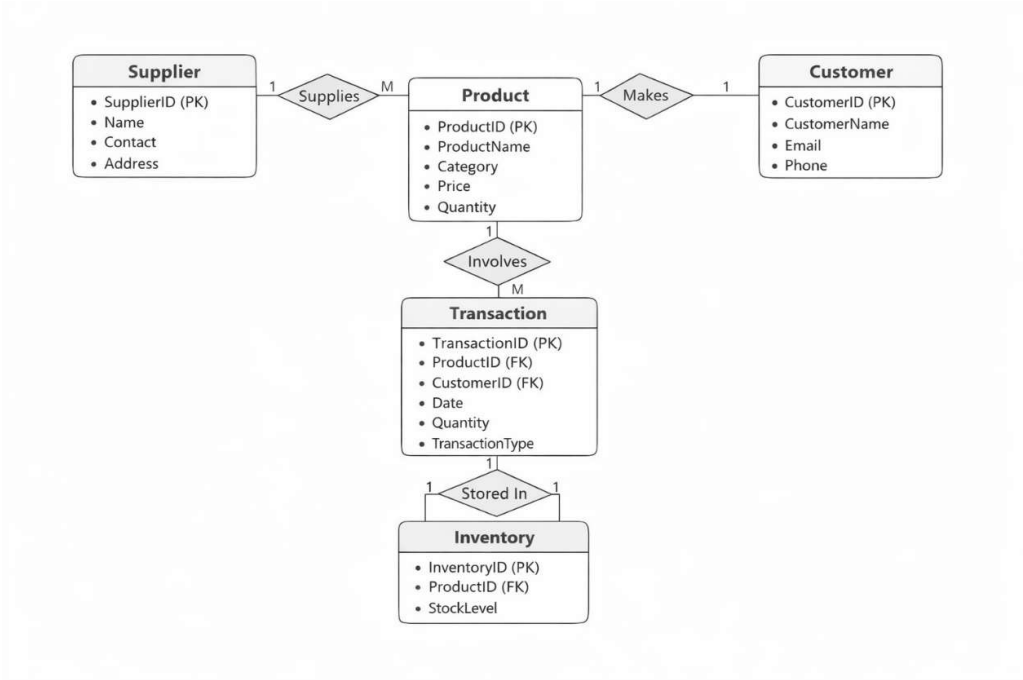


Fig 3.1 E-R Diagram

3.2 DATABASE DESIGN

The database is named inventory_db and contains five main tables designed in Third Normal Form (3NF).

3.2.1 Table Structure

Table 3.1: Suppliers

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
supplier_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique supplier identifier

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
name	VARCHAR(100)	NOT NULL	Supplier company name
contact	VARCHAR(50)	-	Contact phone number
address	VARCHAR(255)	-	Physical address
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

Table 3.2: Customers

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
customer_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique customer identifier
customer_name	VARCHAR(100)	NOT NULL	Customer full name
email	VARCHAR(100)	UNIQUE	Email address
phone	VARCHAR(20)	-	Contact number
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

Table 3.3: Products

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
product_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique product identifier
product_name	VARCHAR(100)	NOT NULL	Product name
category	VARCHAR(50)	-	Product category
price	DECIMAL(10,2)	NOT NULL, CHECK (price >= 0)	Unit price in rupees
quantity	INT	NOT NULL DEFAULT 0	Available quantity
supplier_id	INT	FOREIGN KEY → suppliers(supplier_id)	Reference to supplier
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

Table 3.4: Transactions

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
transaction_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique transaction identifier
product_id	INT	NOT NULL, FOREIGN KEY → products(product_id)	Product involved
customer_id	INT	NOT NULL, FOREIGN KEY → customers(customer_id)	Customer involved
date	DATE	NOT NULL	Transaction date
quantity	INT	NOT NULL, CHECK (quantity > 0)	Quantity transacted
transaction_type	ENUM	NOT NULL	Sale/Purchase/Return/Transfer
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

Table 3.5: Inventory

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
inventory_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique inventory record ID
product_id	INT	NOT NULL, UNIQUE, FOREIGN KEY → products(product_id)	Reference to product
stock_level	INT	NOT NULL DEFAULT 0	Current stock count
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE	Last update timestamp

3.2.2 Keys Summary

Primary Keys:

- **supplier_id** – suppliers table
- **customer_id** – customers table
- **product_id** – products table
- **transaction_id** – transactions table

- **inventory_id** – inventory table

Foreign Keys:

- **products.supplier_id → suppliers.supplier_id** (ON DELETE SET NULL, ON UPDATE CASCADE)
- **transactions.product_id → products.product_id** (ON DELETE RESTRICT, ON UPDATE CASCADE)
- **transactions.customer_id → customers.customer_id** (ON DELETE RESTRICT, ON UPDATE CASCADE)
- **inventory.product_id → products.product_id** (ON DELETE CASCADE, ON UPDATE CASCADE)

Unique Keys:

- **customers.email** – prevents duplicate email addresses

CHAPTER 4

PROJECT METHODOLOGY

4.1 WORKFLOW OF THE SYSTEM

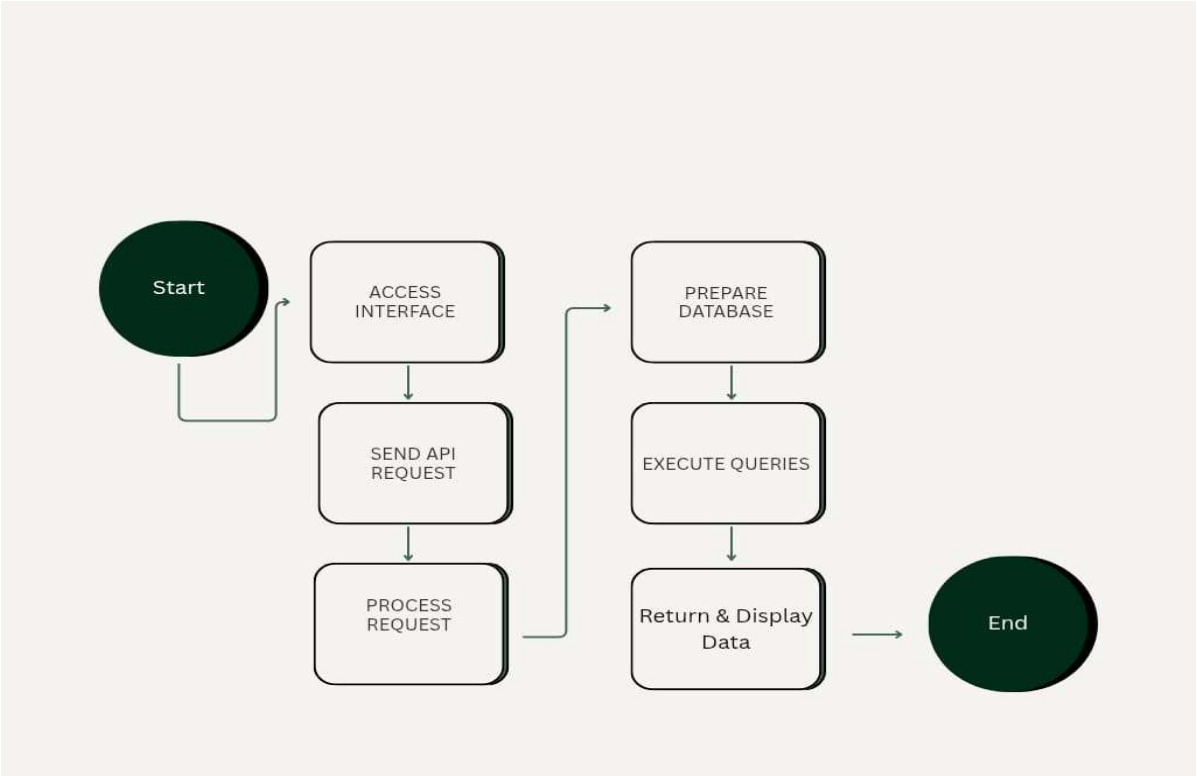


Fig 4.1 Workflow Diagram

This flow diagram represents the working process of a system that retrieves and displays data using an API and database. The process begins with the Start node, where the user initiates the system. The system then accesses the interface, allowing the user to interact with it. Next, an API request is sent to the server based on user input. The request is processed, and necessary actions are determined. The system then prepares the database by establishing connections and organizing required data. Queries are executed to fetch or manipulate data.

CHAPTER 5

SYSTEM REQUIREMENTS

5.1 HARDWARE REQUIREMENTS

The system requirements are defined to ensure both basic functionality and optimal performance for users. At a minimum level, the processor should be an Intel Core i3 or an equivalent AMD processor, which is capable of handling standard operations and basic multitasking. However, for smoother performance and better efficiency, a recommended configuration includes an Intel Core i5 or a higher processor, which significantly improves speed and responsiveness, especially when running multiple applications simultaneously. In terms of memory, the minimum requirement is 4 GB of RAM, which is sufficient for basic usage, but upgrading to 8 GB or more is highly recommended to support advanced tasks, improve system stability, and enhance multitasking capabilities. Storage also plays a crucial role, where a minimum of 100 GB HDD is required to store essential files and software, while a 256 GB SSD is recommended for faster data access, quicker boot times, and overall improved system performance. The display requirements include a minimum resolution of 1366 × 768, which provides acceptable visual clarity for general use, but a higher resolution of 1920 × 1080 is recommended for a better viewing experience, sharper images, and improved productivity, especially for tasks involving detailed visuals. Finally, network connectivity requires at least a 100 Mbps connection to ensure basic internet functionality, while Gigabit Ethernet is recommended for faster and more reliable network performance, particularly in environments that demand high-speed data transfer and stable connectivity. Overall, while the minimum requirements ensure that the system can operate effectively, the recommended specifications provide a significantly enhanced user experience.

5.2 SOFTWARE REQUIREMENTS

The system software requirements define the essential technologies needed for development and deployment. The operating system can be Windows 10 or 11, Linux distributions such as Ubuntu, or macOS, providing flexibility across different platforms. For the web server, Apache is required, which can be set up using environments like XAMPP, WAMP, or LAMP depending on the operating system. The database requirement is MySQL version 5.7 or higher, ensuring reliable data storage and management capabilities. The backend development relies on PHP version 7.4 or above, which offers improved performance and modern features. On the frontend side, technologies such as HTML5, CSS3, and JavaScript (ES6 or later) are used to build responsive and interactive user interfaces. For accessing and testing the application, modern web browsers like Google Chrome, Mozilla Firefox, or Microsoft Edge in their latest versions are recommended to ensure compatibility and security. Additionally, development tools such as Visual Studio Code or any standard text editor can be used to write and manage the code efficiently. Together, these specifications ensure a stable, flexible, and efficient environment for both development and execution of the system.

XAMPP Stack Configuration:

- Apache Port: 80
- MySQL Port: 3306
- PHP Version: 8.x
- phpMyAdmin for database management

CHAPTER 6

MODULE DESCRIPTION

6.1 USER MODULE

The User Module provides the interface for interacting with the system through the web browser. Users can perform the following operations:

Features:

- View dashboard with key metrics and analytics
- Browse and search products, suppliers, and customers
- Create new transactions (sales, purchases, returns, transfers)
- Monitor inventory status with visual indicators
- Generate reports on demand

User Actions:

The system includes several user actions designed to improve navigation and data handling efficiency. The navigate action allows users to switch seamlessly between different sections such as Dashboard, Products, Suppliers, Customers, Transactions, and Inventory pages, ensuring smooth access to all modules. The search functionality enables users to filter and locate specific records within tables using search bars, making data retrieval faster and more convenient. The view action is used to display detailed information in a structured tabular format, helping users clearly understand and analyze the data. Additionally, the export feature, which is planned as a future enhancement, will allow users to export data into formats such as CSV or PDF for reporting and

documentation purposes. Together, these actions enhance usability, improve user experience, and support efficient system interaction.

6.2 ADMIN MODULE

The Admin Module provides full CRUD capabilities, allowing efficient management of system data. For suppliers, the admin can add new suppliers with details such as name, contact, and address, view all suppliers along with their product count, update supplier information, and remove supplier records when necessary. In the customer section, the admin can register new customers, view a list of customers along with their transaction counts, update customer details, and delete customer records. The product management feature enables the admin to add products with information like category, price, quantity, and supplier, view the product catalog with current stock levels, modify product details, and remove products when required. For transactions, the admin can record different types such as sales, purchases, returns, or transfers, view the complete transaction history, and delete transaction records if needed, although updating transactions is not applicable.

- **Supplier** – **Create:** Add new supplier with name, contact, and address, **Read**: View all suppliers with product count, **Update**: Edit supplier details, **Delete:** Remove supplier record.
- **Customer – Create**: Register new customer**, Read**: View customer list with transaction count, **Update:** Update customer information, **Delete**: Delete customer.
- **Product – Create:** Add product with category, price, quantity, and supplier, **Read:** View product catalog with stock levels, **Update:** Modify product details, **Delete:** Remove product**.**
- **Transaction – Create:** Record sale, purchase, return, or transfer, **Read:** View transaction history, **Update:** Not applicable, **Delete:** Delete transaction.

Admin Privileges:

- Full access to all modules
- Ability to modify any record
- Delete permissions with confirmation prompt

6.3 DATABASE MODULE

The Database Module manages all data persistence operations:

Database Schema:

-- Core tables with relationships

suppliers (supplier_id PK)

customers (customer_id PK)

products (product_id PK, supplier_id FK)

transactions (transaction_id PK, product_id FK, customer_id FK)

inventory (inventory_id PK, product_id FK)

Key Operations:

1. **Connection Management** – getDB() function creates connection and auto-creates database/tables if missing
2. **Query Execution** – Prepared statements prevent SQL injection

```
$stmt = $pdo->prepare("INSERT INTO suppliers (name, contact, address) VALUES (?, ?, ?)");

$stmt->execute([$name, $contact, $address]);
```

3. Transaction Management – ACID Compliance for stock updates

```
$pdo->beginTransaction();

// Multiple update operations

$pdo->commit();
```

6.4 AUTHENTICATION MODULE

Note: Current implementation focuses on database structure. Authentication can be added as future enhancement.

Current security features:

- Input sanitization using htmlspecialchars() and strip_tags()
- Prepared statements for all SQL queries
- Server-side validation of required fields
- CORS headers configured

6.5 REPORT GENERATION MODULE

The Report Generation Module provides real-time analytics:

Dashboard Reports:

1. **Summary Statistics**
    - Total products count
    - Total suppliers count
    - Total customers count

- Total transactions count
- Low stock alerts count
- Total revenue (from all sales)

2. **Recent Transactions** (Last 5 records)
    - Customer name
    - Product name
    - Transaction type
    - Total amount
3. **Monthly Revenue Chart**
    - Last 6 months sales data
    - Visual bar chart representation

Inventory Status Report:

Status	**Condition**	**Visual Indicator**
In Stock	stock_level ≥ 30	Green badge
Low Stock	0 < stock_level < 30	Orange badge
Out of Stock	stock_level = 0	Red badge

Sample SQL for Revenue Calculation:

```
SELECT COALESCE(SUM(p.price * t.quantity), 0)

FROM transactions t

JOIN products p ON t.product_id = p.product_id

WHERE t.transaction_type = 'Sale'
```

CHAPTER 7

IMPLEMENTATION

7.1 DATABASE CREATION

The database is automatically created when the application first runs. The db.php file contains the logic:

```
function getDB(): PDO {
   // Connect without database first
   $pdoInit = new PDO("mysql:host=localhost;charset=utf8mb4", "root", "");
   // Create database if not exists
   $pdoInit->exec("CREATE DATABASE IF NOT EXISTS inventory_db");
   // Connect to the database
   $pdo = new PDO("mysql:host=localhost;dbname=inventory_db;charset=utf8mb4", "root", "");
   // Create tables
   createTables($pdo);
   return $pdo;
}
```

7.2 SQL QUERIES

7.2.1 INSERT Operation

Adding a new supplier:

```
INSERT INTO suppliers (name, contact, address)
VALUES ('TechSource Pvt Ltd', '9876543210', '12, MG Road, Bengaluru');
```

Adding a new product (with inventory):

```
-- Start transaction
START TRANSACTION;
INSERT INTO products (product_name, category, price, quantity, supplier_id)
VALUES ('Laptop Dell XPS 15', 'Electronics', 85000.00, 50, 1);
INSERT INTO inventory (product_id, stock_level)
VALUES (LAST_INSERT_ID(), 50);
COMMIT;
```

7.2.2 SELECT Operation

List all products with supplier and stock status:

```
SELECT p.*, s.name AS supplier_name,
i.stock_level,
```

```
    CASE
        WHEN i.stock_level = 0 THEN 'Out of Stock'
        WHEN i.stock_level < 30 THEN 'Low Stock'
        ELSE 'In Stock'
    END AS stock_status
FROM products p
LEFT JOIN suppliers s ON p.supplier_id = s.supplier_id
LEFT JOIN inventory i ON p.product_id = i.product_id
ORDER BY p.product_id;
```

Monthly revenue report:

```
SELECT DATE_FORMAT(t.date, '%b %Y') AS month,
    SUM(p.price * t.quantity) AS revenue
FROM transactions t
JOIN products p ON t.product_id = p.product_id
WHERE t.transaction_type = 'Sale'
GROUP BY DATE_FORMAT(t.date, '%b %Y')
ORDER BY t.date DESC
LIMIT 6;
```

7.2.3 UPDATE Operation

Update product price and details:

```
UPDATE products

SET product_name = ?, category = ?, price = ?, quantity = ?, supplier_id = ?

WHERE product_id = ?;
```

Update inventory after transaction:

```
-- For Sale (decrease stock)

UPDATE inventory SET stock_level = stock_level - ? WHERE product_id = ?;

-- For Purchase (increase stock)

UPDATE inventory SET stock_level = stock_level + ? WHERE product_id =
?;
```

7.2.4 DELETE Operation

```
-- Delete supplier (products will have supplier_id set to NULL)

DELETE FROM suppliers WHERE supplier_id = ?;

-- Delete product (inventory record automatically deleted due to CASCADE)

DELETE FROM products WHERE product_id = ?;
```

7.3 CONSTRAINT

7.3.1 PRIMARY KEY

Ensures each record is uniquely identifiable:

```
CREATE TABLE suppliers (
   supplier_id INT AUTO_INCREMENT PRIMARY KEY,
   ...
);
```

Purpose: Uniquely identifies each supplier, customer, product, transaction, and inventory record.

7.3.2 FOREIGN KEY

Maintains referential integrity between related tables:

```
-- Products reference suppliers
FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id)
   ON DELETE SET NULL    -- If supplier deleted, set to NULL
   ON UPDATE CASCADE     -- If supplier ID changes, update automatically
-- Transactions reference products and customers
FOREIGN KEY (product_id) REFERENCES products(product_id)
   ON DELETE RESTRICT    -- Prevent deletion if transactions exist
```

```
    ON UPDATE CASCADE

-- Inventory references products

FOREIGN KEY (product_id) REFERENCES products(product_id)

    ON DELETE CASCADE     -- If product deleted, delete inventory record

    ON UPDATE CASCADE
```

7.3.3 NOT NULL

Ensures required fields always contain values:

```
CREATE TABLE products (

    product_name VARCHAR(100) NOT NULL,

    price DECIMAL(10,2) NOT NULL DEFAULT 0.00,

    ... );
```

7.3.4 UNIQUE

Prevents duplicate values in specific columns:

```
CREATE TABLE customers (

    email VARCHAR(100) UNIQUE,

    ...

)
```

CHAPTER 8

CONCLUSION

CONCLUSION :

The Inventory Management System developed as part of this project successfully demonstrates the effective application of Database Management System concepts to solve real-world inventory tracking challenges.

Key Achievements:

1. **Structured Database Design** – A normalized relational schema with five interconnected tables ensures data integrity and eliminates redundancy.

2. **Real-time Stock Management** – Inventory levels automatically update after each transaction (sale, purchase, return, transfer), providing accurate current stock information.

3. **Comprehensive CRUD Operations** – Full create, read, update, delete functionality for suppliers, customers, products, and transactions.

4. **Intelligent Stock Status** – Visual indicators (In Stock, Low Stock, Out of Stock) help identify products requiring attention.

5. **Analytics Dashboard** – Real-time statistics including total revenue, low stock alerts, and monthly sales trends.

6. **Automated Database Setup** – The system automatically creates the database and tables on first run, simplifying deployment.

7. **Web-based Interface** – Modern, responsive design accessible from any device with a browser.

Benefits Realized:

The system addresses several key challenges by implementing effective solutions to improve performance and reliability. Data redundancy is minimized through a normalized database design, which ensures that duplicate data is avoided and consistency is maintained. Issues related to inconsistent records are resolved by applying foreign key constraints and proper transaction management, ensuring data accuracy and integrity. Slow data retrieval is improved by using optimized SQL queries along with JOIN operations, which enhance query efficiency and speed. Manual stock updates are eliminated by introducing automatic inventory adjustment, reducing human effort and minimizing errors. Poor reporting capabilities are enhanced by providing a real-time dashboard and analytics, allowing users to monitor and analyze data effectively. Additionally, limited accessibility is overcome by developing a web-based interface, enabling users to access the system from anywhere with ease.

- **Data Redundancy –** Solved using a normalized database design to eliminate duplicate data.
- **Inconsistent Records –** Handled using foreign key constraints and transaction management.
- **Slow Retrieval –** Improved using optimized SQL queries with JOIN operations.
- **Manual Stock Updates –** Eliminated through automatic inventory adjustment mechanisms.
- **Poor Reporting –** Enhanced with real-time dashboard and analytics features.
- **Limited Accessibility –** Overcome by implementing a web-based interface.

Future Enhancements:

1. **User Authentication** – Role-based access control (Admin, Manager, Staff)
2. **Barcode/RFID Integration** – Faster product lookup and stock updates
3. **Email Alerts** – Automatic notifications for low stock products
4. **Export Functionality** – Download reports as PDF, Excel, or CSV
5. **Purchase Order Management** – Generate and track purchase orders
6. **Sales Forecasting** – Predictive analytics based on historical data
7. **Mobile Application** – Dedicated mobile app for inventory management

APPENDIX A: SOURCE CODE STRUCTURE

A.1 Project File Organization

A.2 Key Code Snippets

Database Connection (includes/db.php):

```
<?php
define('DB_HOST', 'localhost');
define('DB_NAME', 'inventory_db');
define('DB_USER', 'root');
define('DB_PASSWORD', '');
function getDB(): PDO {
   $pdo = new PDO(
      "mysql:host=" . DB_HOST . ";dbname=" . DB_NAME . ";charset=utf8mb4",
      DB_USER,
      DB_PASSWORD,
      [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
   );
   return $pdo;
}
 ?>
```

API Route Handler (api.php excerpt):

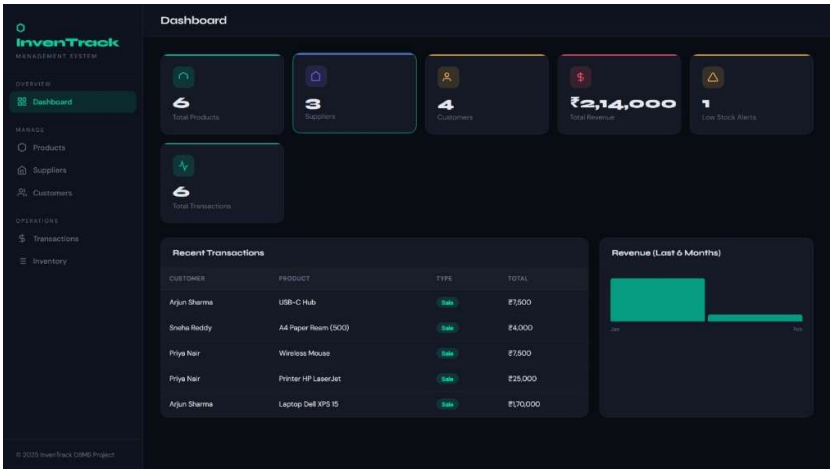
```
<?php
header('Content-Type: application/json');
require_once __DIR__ . '/includes/db.php';
$module = $_GET['module'] ?? '';
$action = $_GET['action'] ?? 'list';
if ($module === 'products') {
    if ($action === 'list') {
        $rows = $pdo->query("SELECT * FROM products")->fetchAll();
        jsonResponse(['success' => true, 'data' => $rows]);
    }
    // ... other actions
}
?>
```

Frontend API Call (index.html excerpt):

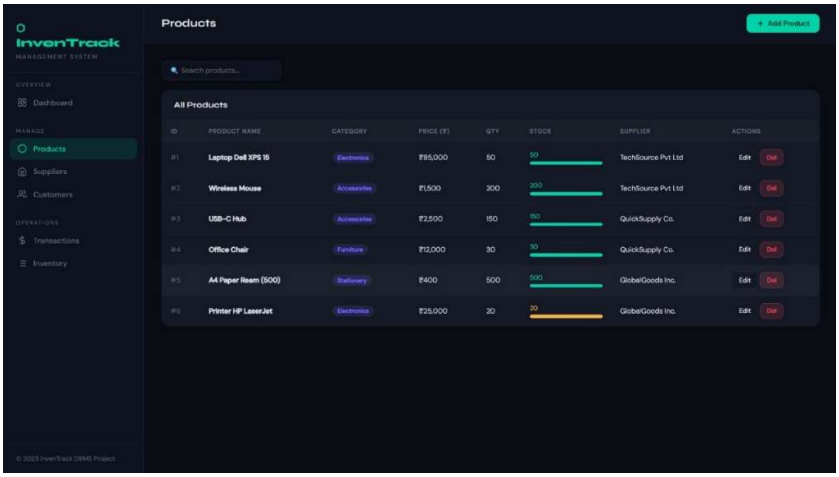
```
async function loadProducts() {
    const response = await fetch('api.php?module=products&action=list');
    const data = await response.json();
    // Render products table
}
```

APPENDIX B: SCREENSHOTS

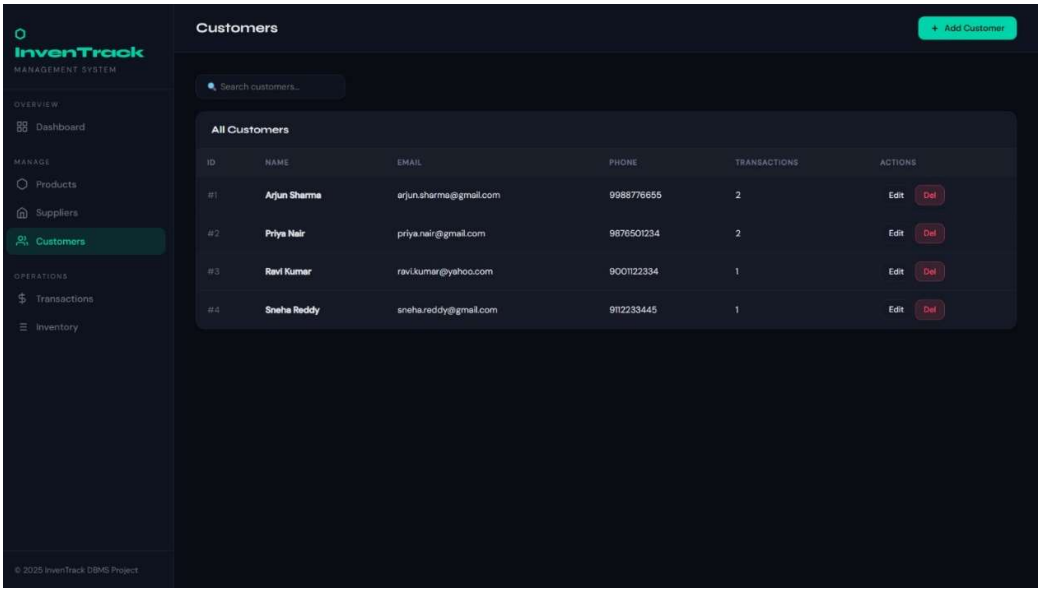
B.1 Dashboard View



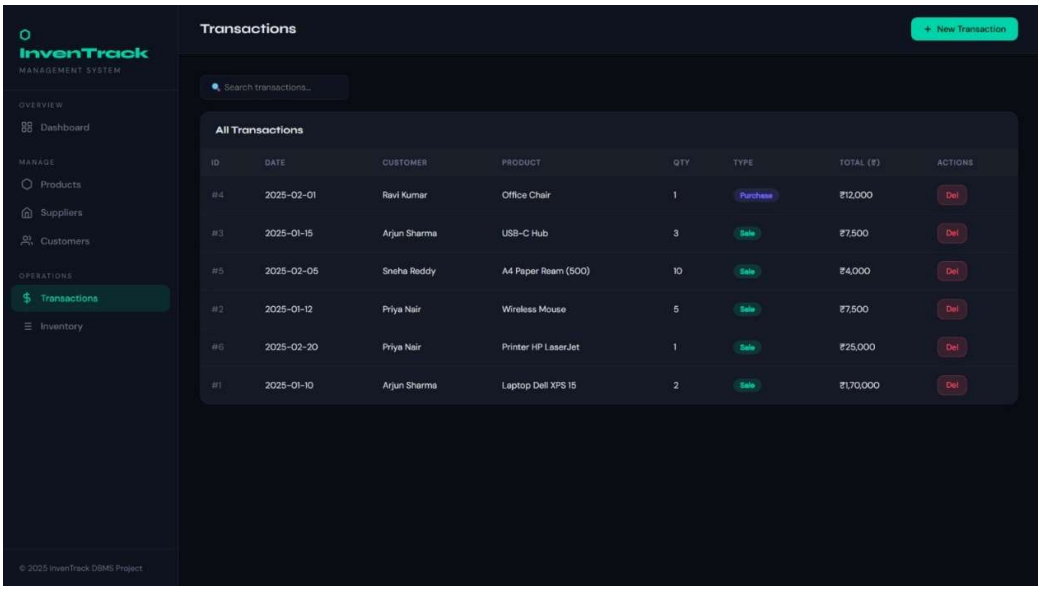
B.2 Products Management Page



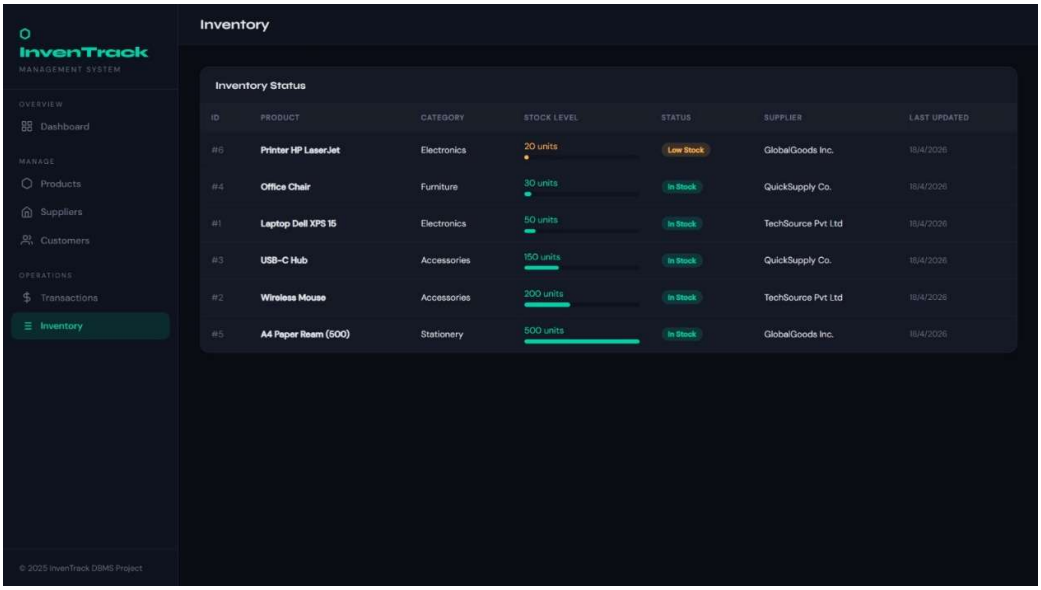
B.3 Customers Management Page



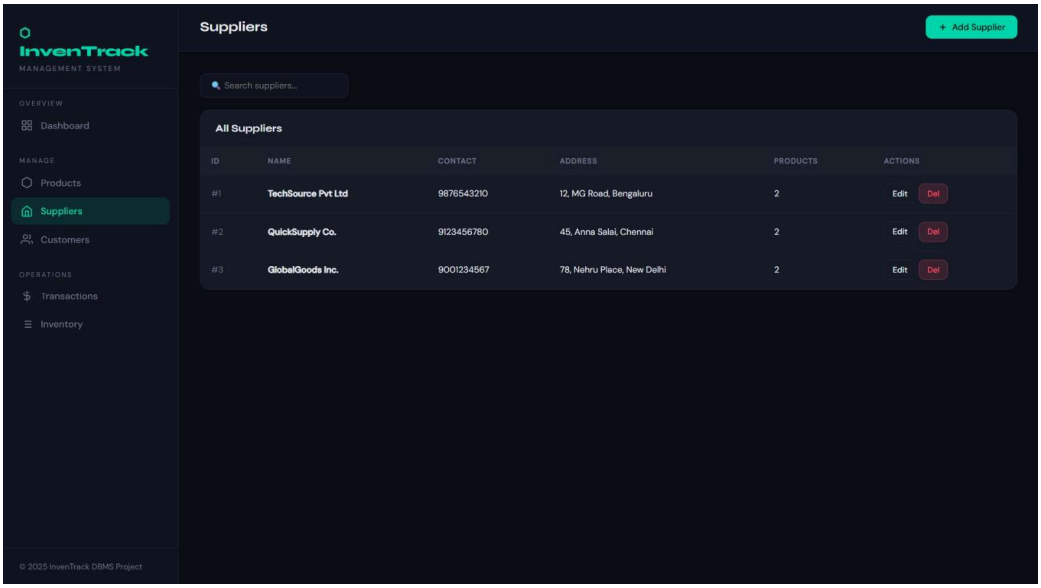
B.4 Transactions Page



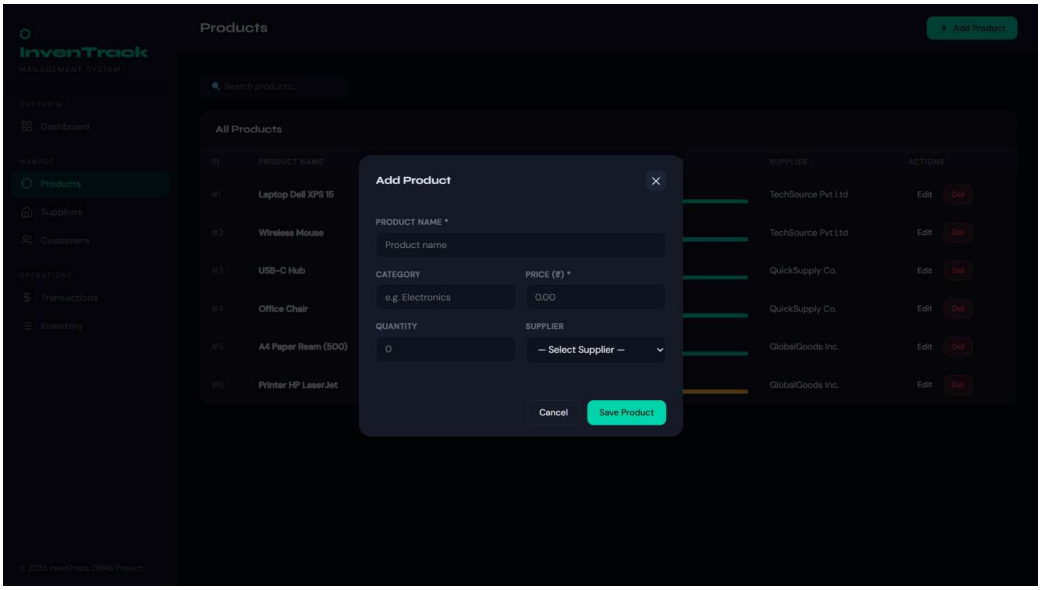
B.5 Inventory Page

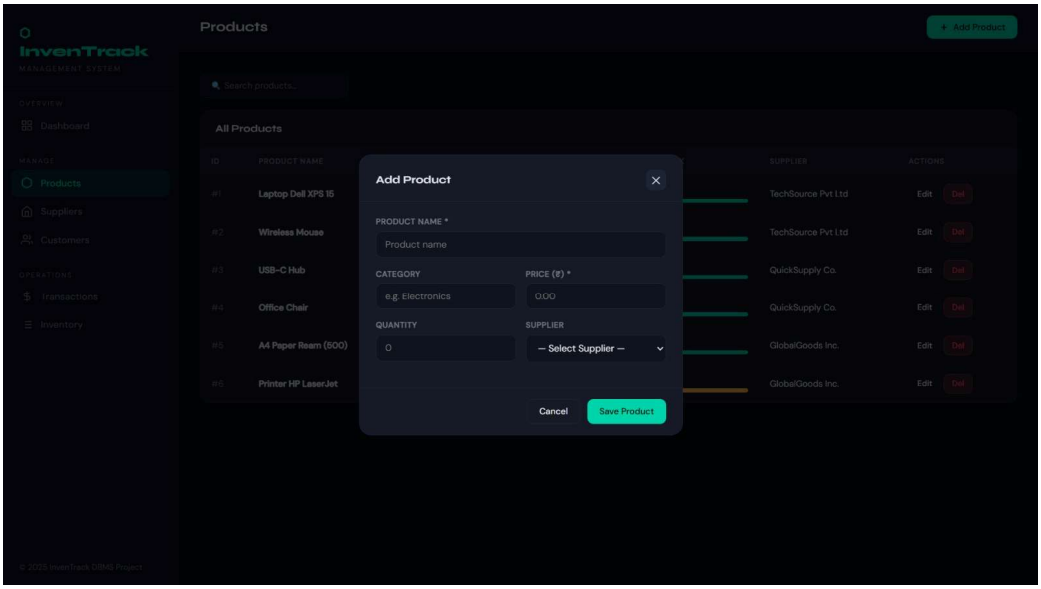


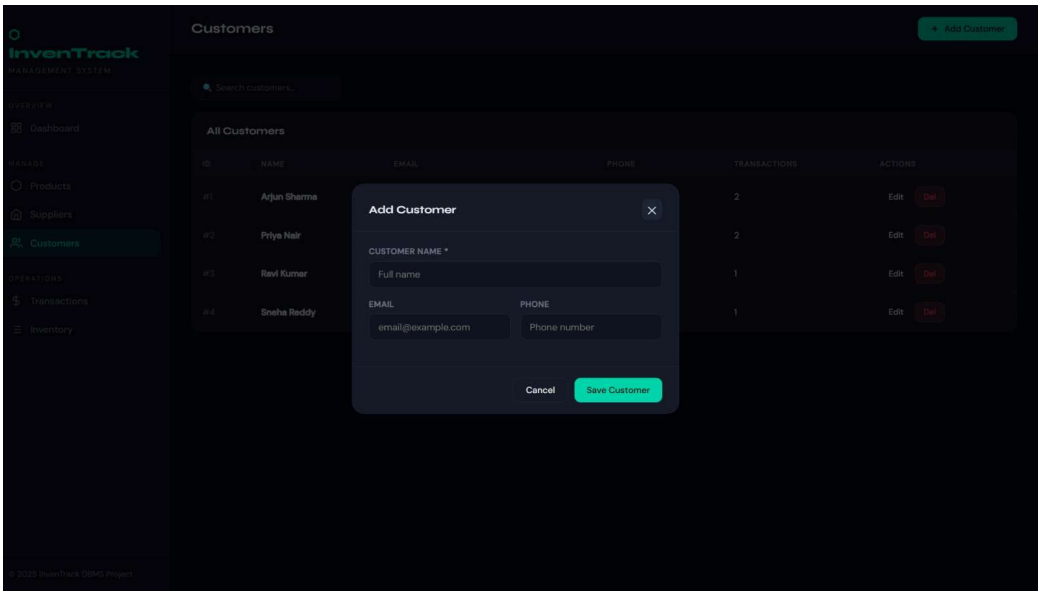
B.6 Suppliers page

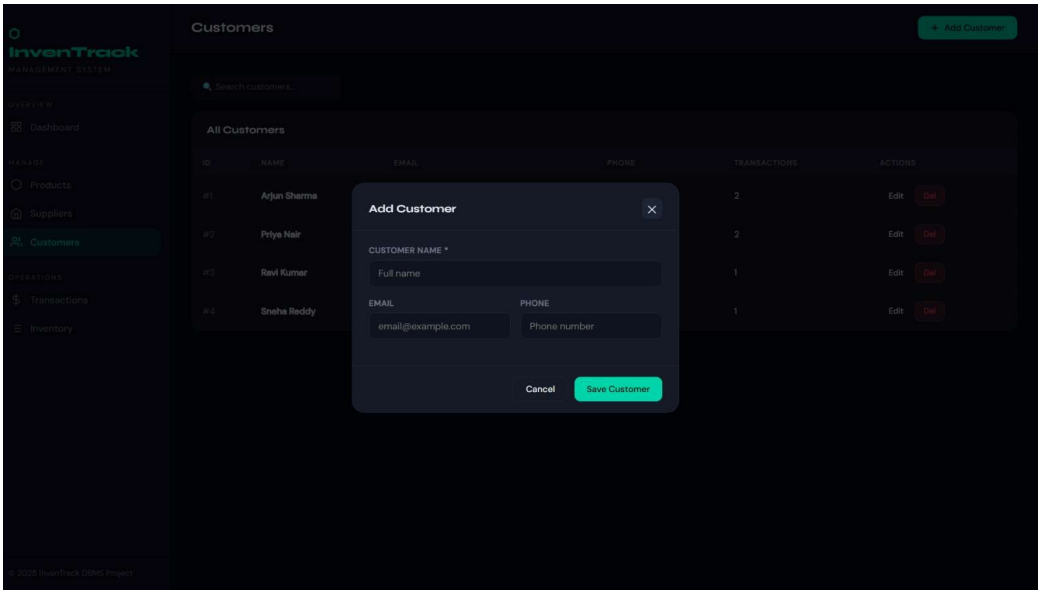


B.7 Add/Edit Forms






InvenTrack
MANAGEMENT SYSTEM
OVERVIEW
Dashboard
MANAGE
Products
Suppliers
Customers
OPERATIONS
Transactions
Inventory
© 2025 InvenTrack DBMS Project
Customers
+ Add Customer
Search customers...
All Customers
ID
NAME
EMAIL
PHONE
TRANSACTIONS
ACTIONS
#1
Arjun Sharma
2
Edit
Del
#2
Priya Nair
2
Edit
Del
#3
Ravi Kumar
1
Edit
Del
#4
Sneha Reddy
1
Edit
Del
Add Customer
CUSTOMER NAME *
Full name
EMAIL
email@example.com
PHONE
Phone number
Cancel
Save Customer


InvenTrack
MANAGEMENT SYSTEM
OVERVIEW
Dashboard
MANAGE
Products
Suppliers
Customers
OPERATIONS
Transactions
Inventory
© 2025 InvenTrack DBMS Project
Customers
+ Add Customer
Search customers...
All Customers
ID
NAME
EMAIL
PHONE
TRANSACTIONS
ACTIONS
#1
Arjun Sharma
2
Edit
Del
#2
Priya Nair
2
Edit
Del
#3
Ravi Kumar
1
Edit
Del
#4
Sneha Reddy
1
Edit
Del
Add Customer
CUSTOMER NAME *
Full name
EMAIL
email@example.com
PHONE
Phone number
Cancel
Save Customer

REFERENCES

1. Naik, Shefali. (2013). *Concepts of Database Management System*.
2. Singh, S. K. (2009). *Database Systems: Concepts, Design and Applications*.
3. Kahate, Atul. (2004). *Introduction to Database Management Systems*.
4. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2010). *Database System Concepts*.
5. Date, C. J. (2004). *An Introduction to Database Systems*.
6. Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems*. McGraw-Hill.
7. Inflibnet Centre. (n.d.). *Introduction to Database Management System*.
8. Naik, Shefali. (2024). *Concept of Database Management System*.
9. Rob, Peter & Coronel, Carlos. (2007). *Database Systems: Design, Implementation, and Management*. Thomson Learning.
10. O'Neil, Patrick & O'Neil, Elizabeth. (2001). *Database Principles, Programming, and Performance*.